

Übung 7: Funktionale Programmierkonzepte in der Projektentwicklung

Team 6 (Thema E-Vote) - Abgabe:

Link: https://github.com/jacque-schu/MSE_eVote

1. Geeignete Stellen finden

Im Bounded Context „Bürgerverwaltung“ lassen sich eventuell diese vier Bereiche für funktionale Programmierung nutzen:

- `validate_name` in `Buerger` (Domain Model: `buerger.py`): Mehrere Validierungsregeln in einer Methode.

```
@field_validator("name")  & frit2x
@classmethod
def validate_name(cls, name: str) -> str:
    if not name or len(name) < 1:
        raise ValueError("Name ist zu kurz.")
    if not re.fullmatch(pattern: r"[A-Za-zÀ-ÿ\s-]+", name):
        raise ValueError("Name enthält ungültige Zeichen.")
    if name.startswith('-') or name.endswith('-'):
        raise ValueError("Name darf nicht mit Bindestrich beginnen oder enden.")
    return name
```

- `lade_alle` / `speichere_alle` im `BuergerRepository` (Infrastruktur: `buerger_repository.py`): Mischung aus I/O und Daten-Transformation.

```
def lade_alle(self) -> List[Buerger]: 5 usages  ⌘ frit2x
    try:
        with open(self.dateipfad, "r", encoding="utf-8") as file:
            daten = json.load(file)
            if isinstance(daten, list):
                return [Buerger(**buerger) for buerger in daten]
            else:
                return []
    except FileNotFoundError:
        return []
    except json.JSONDecodeError:
        # Datei zurücksetzen bei Fehler
        self.speichere_alle([])
        return []
```

```
def speichere_alle(self, buerger_liste: List[Buerger]): 3 usages  ⌘ frit2x
    def date_serializer(obj):  ⌘ frit2x
        if isinstance(obj, date):
            return obj.isoformat()
        raise TypeError(f"Type {type(obj)} not serializable")

    with open(self.dateipfad, "w", encoding="utf-8") as file:
        json.dump([b.model_dump() for b in buerger_liste], file, default=date_serializer,
```

- `fuege_hinzu` im `BuergerRepository` (Infrastruktur: `buerger_repository.py`): Liste laden, mutieren, speichern.

```
def fuege_hinzu(self, neuer_buerger: Buerger): 2 usages  ⌘ frit2x
    buerger_liste = self.lade_alle()
    # Einfach anhängen, keine Duplikatprüfung hier (Optional in Application Service)
    buerger_liste.append(neuer_buerger)
    self.speichere_alle(buerger_liste)
```

- Token-Handling (`authorization` / `token`) im Endpoint (Application/API-Schicht: `buerger_endpoints.py`).

```

🔗 /api/buergerverwaltung/registrierung
@router.post("/registrierung")  ⚡ frit2x
async def registriere_buerger_api(
    request: Request,
    name: str = Form(...),
    adresse: str = Form(...),
    geburtsdatum: str = Form(...),
    email: str = Form(...),
    authentifizierungsdaten: str = Form(...),
    authorization: str = Header(None) # Liest den Authorization-Header aus
):
    init_buerger_db()

    if not authorization:
        raise HTTPException(status_code=401, detail="Authorization Header fehlt")

```

Typische Stellen mit:

- sequentieller Logik (mehrere Prüfungen / Umwandlungen hintereinander),
- veränderbaren Strukturen (Liste, Header-String),
- und gemischter Verantwortlichkeit (I/O + Logik), die durch funktionale Aufteilung klarer werden.

2. Funktionale Ideen

Funktionaler Ansatz für die jeweilige Stelle:

- `validate_name` (Buerger): Aufsplitten in kleine pure Prüffunktionen und Funktionskomposition (Liste von Checks, die nacheinander ausgeführt werden).
 - `lade_alle` / `speichere_alle`: Reine Transformationsfunktionen (JSON → Python-Modelle und zurück) und klarer map-Stil statt „versteckter“ Logik in I/O-Funktionen.
 - `fuege_hinzu`: Kein in-place Mutieren der Liste, sondern eine pure Funktion, die eine neue Liste zurückgibt.
 - Token-Handling im Endpoint: Reines String-Parsing in eine eigene Funktion auslagern, sodass der Endpoint nur noch Funktionen orchestriert.
-

3. Ist / Soll / Verbesserungen

Stelle A: `validate_name` in `Buerger` (Domain Model)

Aktueller Code (Ist-Zustand)

- In der Methode `validate_name` werden mehrere Regeln direkt nacheinander geprüft: Name darf nicht leer sein, muss ein bestimmtes Regex erfüllen, darf nicht mit - beginnen oder enden. Bei Verstößen wird jeweils eine `ValueError` geworfen. (in [buerger.py](#))

Überarbeitung (Soll-Zustand)

- Aufteilen der Logik in mehrere kleine pure Functions, z.B.:
 - `check_not_empty(name)`,
 - `check_valid_pattern(name)`,
 - `check_hyphen_position(name)`.
- Eine zentrale Funktion oder ein kleiner „Validator“, der eine Liste dieser Funktionen nacheinander aufruft (z.B. `for check in checks: check(name)`).

Verbesserung:

```
# Kleine Prüffunktionen
def ensure_not_empty(name: str) -> str: 1usage new *
    if not name or len(name) < 1:
        raise ValueError("Name ist zu kurz.")
    return name

def ensure_valid_pattern(name: str) -> str: 1usage new *
    if not re.fullmatch(pattern: r"[A-Za-zÄ-ÿ\s-]+", name):
        raise ValueError("Name enthält ungültige Zeichen.")
    return name

def ensure_valid_hyphen(name: str) -> str: 1usage new *
    if name.startswith("-") or name.endswith("-"):
        raise ValueError("Name darf nicht mit Bindestrich beginnen oder enden.")
    return name
```

```
@field_validator("name") new *
@classmethod
def validate_name(cls, name: str) -> str:
    # Funktionale „Pipeline“ aus pure Functions
    checks = [ensure_not_empty, ensure_valid_pattern, ensure_valid_hyphen]

    for check in checks:
        name = check(name)

    return name
```

Dadurch:

- Höhere Lesbarkeit, da jede Funktion genau eine Regel beschreibt.
 - Bessere Testbarkeit, weil jede Regel isoliert getestet werden kann.
 - Wiederverwendbarkeit der Prüf-Logik in anderen Kontexten (z.B. bei einer späteren Bürger-Editierfunktion).
-

Stelle B: `lade_alle` / `speichere_alle` / `fuege_hinzu` im `BuergerRepository`

Aktueller Code (Ist-Zustand)

- `lade_alle` öffnet die JSON-Datei, lädt die Daten, prüft, ob es eine Liste ist, baut daraus mit List-Comprehension `Buerger`-Objekte und behandelt Fehler mit `try/except`. (in `buerger_repository.py`)
- `speichere_alle` nimmt eine Liste von `Buerger`, wandelt diese mit `model_dump()` in Dictionaries und serialisiert sie mit `json.dump`, inkl. eines eingebetteten `date_serializer`. (in `buerger_repository.py`)
- `fuege_hinzu` lädt alle Bürger, hängt den neuen Bürger an die bestehende Liste und speichert dann die gesamte Liste wieder. (in `buerger_repository.py`)

Überarbeitung (Soll-Zustand)

- Eine pure Funktion `parse_buerger_liste(daten) -> list[Buerger]`, die nur die Python-Rohdaten in `Buerger`-Instanzen transformiert (z.B. via List-Comprehension).
- Eine pure Funktion `serialize_buerger_liste(buerger_liste) -> list[dict]`, die nur `Buerger`-Modelle zu JSON-kompatiblen Dicts macht.
- Eine pure Funktion `hinzufuegen(buerger_liste, neuer_buerger) -> list[Buerger]`, die eine neue Liste zurückgibt (z.B. `return buerger_liste + [neuer_buerger]`).
- `lade_alle` und `speichere_alle` beschränken sich auf Datei-I/O und rufen die reinen Transformationsfunktionen auf.

Verbesserungen

```
# Pure Functions (keine I/O)
def parse_buerger_liste(daten) -> List[Buerger]: new *
    if not isinstance(daten, list):
        return []
    return [Buerger(**buerger) for buerger in daten]

def serialize_buerger_liste(buerger_liste: List[Buerger]) -> list[dict]: new *
    return [b.model_dump() for b in buerger_liste]

def date_serializer(obj): new *
    if isinstance(obj, date):
        return obj.isoformat()
    raise TypeError(f"Type {type(obj)} not serializable")

def hinzufuegen(buerger_liste: List[Buerger], neuer_buerger: Buerger) -> List[Buerger]: n
    # keine Mutation der ursprünglichen Liste
    return [*buerger_liste, neuer_buerger]
```

```
# funktionale Transformation
    return parse_buerger_liste(daten)

def speichere_alle(self, buerger_liste: List[Buerger]): 2 usages new *
    daten = serialize_buerger_liste(buerger_liste)
    with open(self.dateipfad, "w", encoding="utf-8") as file:
        json.dump(
            daten,
            file,
            default=date_serializer,
            ensure_ascii=False,
            indent=4,
        )

def fuege_hinzu(self, neuer_buerger: Buerger): new *
    alte_liste = self.lade_alle()
    neue_liste = hinzufuegen(alte_liste, neuer_buerger)
    self.speichere_alle(neue_liste)
```

- Trennung von I/O und Domänenlogik
- Transformationslogik (parse/serialize/hinzufügen) lässt sich unabhängig von Dateien testen
- Klarer, funktionaler Datenfluss (JSON → Python-Struktur → Domainmodell und zurück), weniger Seiteneffekte

Stelle C: Token-Handling im Endpoint

Aktueller Code (ist-Zustand, in Worten)

- Der Endpoint `registriere_buerger_api` ruft `init_buerger_db()`, prüft, ob ein `authorization`-Header vorhanden ist, und versucht dann, mit `authorization.split(" ")` den Token zu extrahieren. Bei `IndexError` wird ein HTTP-Fehler geworfen. (in `buerger_endpoints.py`)
- Die Logik zum Parsing und Validieren des Headers ist direkt im Endpoint eingebettet.

Geplante Überarbeitung (funktionaler Ansatz)

- Einführung einer reinen Hilfsfunktion `extrahiere_bearer_token(header: str) -> str`, die:
 - prüft, ob der Header im Format `"Bearer <token>"` vorliegt,
 - andernfalls eine klar definierte Exception (z.B. `ValueError`) wirft.
- Der Endpoint verwendet nur diese Funktion und wandelt eine eventuell geworfene Exception in eine `HTTPException` um.

Erwartete Verbesserungen

- Endpoint-Code wird schlanker und konzentriert sich auf HTTP-spezifische Aspekte.
 - Token-Parsing ist als pure Function wiederverwendbar (z.B. für andere Endpoints / Tests).
 - Fehlerpfade sind klarer und besser testbar.
-